Danmarks Tekniske Universitet

s257473
s253050
s225786
s253191

02180 - Introduction to AI

Board Game Assignment

Ricochet Robots

23-03-2026

Contents

1 Game Description **2**

1.1 Game rules 2

1.2 Game type 2

1.3 Size of the state space 2

2 Game Elements **3**

3 States and Moves **3**

3.1 Concrete Representation 3

3.2 Search Algorithms and Example 4

4 Parameters and Future Work **4**

5 Benchmarking AIs **4**

6 Heuristic Functions **5**

1 Game Description

1.1 Game rules

Ricochet Robots is a puzzle game played on a 16×16 board with walls and four coloured robots. Each round, a target chip designates a square and a robot colour. Robots slide in a straight line until hitting a wall or another robot; each slide counts as one move. In our implementation, a solution is valid only if the active robot has **changed direction at least once** before reaching the target (ricochet condition).

Other **simplification** we did:

- **Rainbow chip (color swirl)**: This game piece has not been implemented. It functions just like any other game piece, but it can be reached by every robot and not just by one of the same color.
- **Black robot**: This robot has not been implemented. In the actual board game it works as a wild card that a player can use to their advantage to reach the goal piece in fewer moves, or to reach the color swirl piece.
- **Mirrors**: To simplify the game design, mirrors were not added into the code. They function differently to other walls. When a robot hits a wall, it must stop; however, when the robot reaches a mirror "wall", which is placed diagonally on a single board square, the robot must bounce off the mirror and move 90° in the direction the mirror is pointing. The mirror does not cost an extra move.
- **Board**: In the board game, it is possible to change the board by switching the four pieces that compose the board. In this implementation, only one fixed board scenario is used to simplify the code.
- **Timer**: Playing the actual board game, each player gets a set amount of time to think about their move. In the coded version, there are different AIs playing the same scenario (game board), thus, we assess the time that each AI takes to find the solution. For that reason, a set timer has not been implemented.

1.2 Game type

The game is **deterministic**, **perfect-information**, and **turn-based**. For the AI, it is modeled as a **single-agent planning problem**: one decision-maker controls all robots and minimizes the number of moves. Classical state-space search (BFS, DFS, A*) is therefore appropriate.

1.3 Size of the state space

With 4 robots on a 16×16 board: $256 \times 255 \times 254 \times 253 \approx 4.16 \times 10^9$ positional configurations. Including the ricochet flag (Section 3), the total state space exceeds 8×10^{10}.

The effective reachable space is substantially smaller due to wall constraints, motivating informed search.

2 Game Elements

- $R = \{r_{\text{red}}, r_{\text{blue}}, r_{\text{green}}, r_{\text{yellow}}\}$: robots.
- $C = \{(x, y) \mid 1 \leq x, y \leq 16\}$: board cells.
- W: set of walls (board edges and internal obstacles).
- $T \in C \times R$: target (cell, robot colour).
- $\rho \in \{\textit{true}, \textit{false}\}$: ricochet flag, whether the active robot has already changed direction.

3 States and Moves

s_0**:** (P_0, ρ_0) where P_0 is a random valid placement of all robots (no overlap, not on target or centre block) and $\rho_0 = \textit{false}$.

State s**:** (P, ρ) where $P = (p_1, \ldots, p_4)$, $p_i \in C$. Two states with identical P but different ρ are distinct.

Actions: (r, d) with $r \in R$, $d \in \{N, S, E, W\}$, legal only if the robot moves.

Transition: Deterministic slide until wall or robot; ρ becomes *true* if the active robot changes direction.

Goal Test: $\text{pos}(r) = T$ **and** $\rho = \textit{true}$.

Utility: Number of moves (minimised).

Player: Single agent controlling all robots.

3.1 Concrete Representation

States are Python dicts `{color: (row,col)}`, hashable via `tuple(sorted(robots.items()))`. The ricochet flag is recomputed on demand from the move history (list of `(color, direction)` pairs). The state key is:

```
(tuple(sorted(robots.items())), ricocheted(history, active))
```

The AI interacts exclusively through the `Game` public API (`get_successors`, `is_goal`, `state_key`, `heuristic_i`), ensuring clean separation from the game engine.

3.2 Search Algorithms and Example

BFS expands level by level — optimal but slow (exponential node growth). **DFS** explores deeply with a stack — fast but neither optimal nor complete with a depth cap. **A*** uses $f(n) = g(n) + h(n)$; four heuristics are compared (Section 6).

Example. Red robot R at $(6, 1)$, target $T = (6, 8)$, blue robot B at $(6, 5)$. A*4 search (first levels):

Node	State (P, ρ)	g	h_4	f
n_0	$R = (6, 1),\ \rho = F$	0	3	3
n_1	$R = (6, 4),\ \rho = F$ [RW]	1	2	3
n_2	$R = (3, 1),\ \rho = F$ [RN]	1	3	4
n_4	$R = (6, 4),\ \rho = T$ [RW,RN,RS]	3	1	4
n_5	$R = (6, 8),\ \rho = T$ [...,RE]	4	0	**4**

n_1 is expanded first ($f = 3$). After 4 moves, n_5 satisfies both goal conditions. With H_3, $h_3(n_1) \approx 0.97$ instead of $h_4(n_1) = 2$, causing many extra node expansions.

4 Parameters and Future Work

SOLVER_MAX_MOVES (default 20): depth cap for interactive use.

BENCH_MAX_MOVES (default 8): depth cap for benchmarking, chosen to keep runtime manageable while covering typical puzzle difficulties. A low cap favors strong heuristics (A*4 rarely hits it); BFS suffers more as depth grows.

game.heuristic : when the AI is running, all different heuristics are executed, but if a game is to be played, and parameters are to be fixed, than this parameter would be important to consider in a way that maximizes the performance of the AI implementation.

Future work: relax fewer constraints in H_4 (allow one helper robot to move freely) for a tighter bound; use bidirectional A* to halve effective search depth; larger benchmark (500+ scenarios) with confidence intervals; and conduct a human vs. AI comparison during multiplayer sessions.

5 Benchmarking AIs

20 random scenarios, max 8 moves. Metrics: avg solution length, avg time, avg visited nodes, scenarios solved.

Solver	Avg Moves	Avg Time (s)	Avg Visited	Solved
BFS	5.4	0.981	11 020	20/20
DFS	7.8	0.144	23 411	13/20
A*1	5.4	0.767	6 467	20/20
A*2	5.4	0.403	3 671	20/20
A*3	5.5	0.357	3 149	20/20
A*4	5.4	0.089	187	20/20

BFS is optimal but visits 11 020 nodes on average. DFS is fastest but solves only 13/20 and returns suboptimal paths. A*1–A*3 progressively reduce node expansions by adding richer heuristic signals. A*3 occasionally returns a suboptimal solution (avg 5.5 vs. 5.4), suggesting slight inadmissibility in practice. **A*4** visits only 187 nodes on average — 17× fewer than A*3, 59× fewer than BFS — while remaining optimal and being the fastest solver (0.089 s).

6 Heuristic Functions

All A* variants use $f(n) = g(n) + h(n)$. Let r be the active robot, T the target. Define:

$$\text{aligned}(r, T) = \big(\text{row}(r) = \text{row}(T)\big) \vee \big(\text{col}(r) = \text{col}(T)\big),$$

$$\text{ricocheted}(r, \pi) = r \text{ changed direction at least once in } \pi$$

Note. Two intermediate heuristics H_4 and H_5 were tested. H_4 checks reachability in one slide (discrete $\{0, 1, 2\}$); $H_5 = \max(H_3, H_4)$. Neither improved on H_3 — H_4 discriminates poorly, and $H_5 \approx H_3$ in practice. Both are dominated by H_4 and omitted below.

H1 — Alignment + Ricochet.

$$h_1(n) = \begin{cases} 0 & \text{if pos}(r) = T \wedge \text{ricocheted} \\ 1 & \text{if aligned} \wedge \text{ricocheted} \\ 2 & \text{otherwise} \end{cases}$$

Admissibility: aligned+ricocheted needs 1 more move; otherwise ≥ 2.

H2 — Manhattan + Alignment.

$$h_2(n) = \frac{d(r, T)}{30} + \frac{28}{30} \cdot \big(1 - \mathbf{1}[\text{aligned}]\big)$$

Admissibility: $h_2 \leq 1 + 28/30 < 2 \leq h^*$ in all cases.

H3 — Manhattan + Alignment + Ricochet.

$$h_3(n) = \frac{d(r,T)}{30} + \frac{28}{30} \cdot \big(1 - \mathbf{1}[\text{aligned}]\big) + 1 \cdot \big(1 - \mathbf{1}[\text{ricocheted}]\big)$$

Admissibility: $h_3 < 3 \leq h^*$ in the worst case.

H4 — Relaxed Problem via Mini-BFS. H_4 solves a **relaxed problem**: all other robots are frozen as walls and the ricochet condition is removed. Let $\text{slide}^*(r)$ be the minimum slides in this relaxed problem (computed by BFS, depth capped at 4, $\leq$256 states):

$$h_4(n) = \begin{cases} 0 & \text{if } \text{pos}(r) = T \wedge \text{ricocheted} \\ \text{slide}^*(r) + 1 & \text{if } \neg\text{ricocheted} \wedge \text{slide}^*(r) \leq 1 \\ \text{slide}^*(r) & \text{otherwise} \end{cases}$$

Case 2 adds 1 because if the robot can reach the target in ≤ 1 slide but has not yet ricocheted, at least one extra move is needed. *Admissibility:* the relaxed problem is easier, so $\text{slide}^*(r) \leq h^*(n)$; case 2 adds 1 only when genuinely required. H_4 dominates H_1–H_3 because it returns the *exact* relaxed cost (e.g. 3) where H_3 returns an approximation (≈ 0.97), pruning far more nodes. A cache on (positions, ricochet flag) avoids redundant mini-BFS calls. The per-node cost is higher than H_3, but the 17$\times$ reduction in nodes more than compensates on puzzles of ≥ 5 moves.